

Algoritmi e Strutture Dati – 22/06/2026

Esercizio A1 – Complessità – Punti ≥ 8

È facile vedere che $T(n)$ è $\Omega(n)$, per via della parte non ricorsiva. Verifichiamo se $T(n)$ è anche $\mathcal{O}(n)$.

- **Caso base:** per $n = 1, \dots, 48$, si deve dimostrare che $T(n) = 1 \leq cn$; questa disequazione è vera per qualunque $c \geq 1$, $c \geq 1/2$, $c \geq 1/3$, ..., $c \geq 1/48$ rispettivamente; è quindi vera per ogni $c \geq 1$.
- **Ipotesi induttiva:** ipotizziamo che per ogni valore k , $1 \leq k < n$: $T(k) \leq ck$;
- **Passo induttivo:** consideriamo i casi con $n \geq 49$, per i quali $\lfloor n/49 \rfloor$ è maggiore o uguale a 1 (il primo caso base); applichiamo la sostituzione e otteniamo:

$$\begin{aligned} T(n) &= 2T(\lfloor n/4 \rfloor) + 3T(\lfloor n/9 \rfloor) + 7T(\lfloor n/49 \rfloor) + n \\ &\leq 2c\lfloor n/4 \rfloor + 3c\lfloor n/9 \rfloor + 7c\lfloor n/49 \rfloor + n && \text{Sostituzione} \\ &\leq \frac{1}{2}cn + \frac{1}{3}cn + \frac{1}{7}cn + n && \text{Eliminazione } \lfloor \rfloor \\ &= \frac{41}{42}cn + n \stackrel{?}{\leq} cn && \text{Passaggio algebrico} \end{aligned}$$

L'ultima disequazione è vera per $c \geq 42$.

Abbiamo quindi dimostrato che $T(n) = \mathcal{O}(n)$, per $m = 1$ (derivante dal fatto che abbiamo dimostrato l'induzione a partire dal valore 1) e $c = 42$. Poiché $T(n)$ è anche $\Omega(n)$, abbiamo che $T(n) = \Theta(n)$.

Esercizio A2 – Nodi valore-bilanciati – Punti ≥ 10

L'algoritmo visita l'albero in post-ordine e, per ogni nodo, calcola due informazioni: la somma dei valori contenuti nel sottoalbero radicato in quel nodo e il numero di nodi valore-bilanciati presenti nello stesso sottoalbero. Un nodo è valore-bilanciato se la somma dei valori nel suo sottoalbero sinistro è uguale alla somma dei valori nel suo sottoalbero destro.

La funzione principale `valueBalanced` richiama una procedura ricorsiva `valueBalancedRec` che, dato un nodo t , restituisce una coppia $\langle sum, count \rangle$, dove:

- sum è la somma dei valori di tutti i nodi nel sottoalbero radicato in t ;
- $count$ è il numero di nodi valore-bilanciati contenuti nello stesso sottoalbero.

Se $t = \text{nil}$, il sottoalbero è vuoto: la somma dei valori è pari a 0 e non vi sono nodi valore-bilanciati. Se $t \neq \text{nil}$, la procedura richiama ricorsivamente `valueBalancedRec` sui figli sinistro e destro di t , ottenendo le coppie $\langle sumL, countL \rangle$ e $\langle sumR, countR \rangle$. La somma del sottoalbero radicato in t è quindi

$$sum = sumL + sumR + t.value.$$

Il numero di nodi valore-bilanciati è inizialmente

$$count = countL + countR,$$

e viene incrementato di una unità se e solo se $sumL = sumR$, cioè se il nodo t è valore-bilanciato. La funzione `valueBalanced()` restituisce infine il valore $count$ associato. La complessità è $\Theta(n)$, corrispondente alla visita di un albero.

int valueBalanced(**TREE** T)

int $sum, count = \text{valueBalancedRec}(T)$
return $count$

int, int valueBalancedRec(**TREE** t)

if $t == \text{nil}$ **then**
 return $\langle 0, 0 \rangle$
else
 int $sumL, countL = \text{valueBalancedRec}(t.\text{left}())$
 int $sumR, countR = \text{valueBalancedRec}(t.\text{right}())$
 int $count = countL + countR$
 int $sum = sumL + sumR + t.\text{value}$
 if $sumL == sumR$ **then**
 $count = count + 1$
 return $\langle sum, count \rangle$

Esercizio A3 – Nodi aciclici – Punti ≥ 12

Un nodo s è aciclico se e solo se non esiste un cammino non vuoto che parte da s e ritorna in s . Una soluzione corretta ma ingenua consiste quindi nel ripetere una visita in profondità a partire da ogni nodo s e verificare se, durante la visita, si incontra un arco che torna al nodo di partenza s .

È importante osservare che non è sufficiente verificare se, durante la DFS partita da s , si trova un qualunque ciclo: questo proverebbe solo che da s è raggiungibile un ciclo, non che s appartiene a quel ciclo.

int acyclicNodes(**GRAPH** G)

precondition: $G.n \geq 0$
int $count = 0$
for $s = 0$ **to** $G.n - 1$ **do**
 boolean[] $visited = \text{new boolean}[0 \dots G.n - 1] = \{\text{false}\}$
 if $\text{reachesStart}(G, s, s, visited) == \text{false}$ **then**
 $count = count + 1$
return $count$

boolean reachesStart(**GRAPH** G , **int** u , **int** s , **boolean** [] $visited$)

precondition: $u \in G.V()$ and $s \in G.V()$
 $visited[u] = \text{true}$
foreach $v \in G.\text{adj}(u)$ **do**
 if $v == s$ **then**
 return **true**
 if not $visited[v]$ **then**
 if $\text{reachesStart}(G, v, s, visited)$ **then**
 return **true**
return **false**

La correttezza segue dalla caratterizzazione precedente. Se reachesStart restituisce **true**, allora durante la visita è stato trovato un cammino da s a un nodo u e un arco (u, s) ; quindi s appartiene a un ciclo. Viceversa, se s appartiene a un ciclo, esiste un cammino non vuoto che parte da s e ritorna in s ; la DFS segue gli archi raggiungibili da s e, quando considera l'arco che chiude il ciclo su s , restituisce **true**. Dunque $\text{acyclicNodes}()$ conta esattamente i nodi che non appartengono ad alcun ciclo.

Il vettore *visited* viene reinizializzato a ogni iterazione. Il costo complessivo è quindi $\mathcal{O}(n(m+n)) = \mathcal{O}(mn)$. Questa soluzione ha preso 80%.

Una soluzione più efficiente sfrutta le componenti fortemente connesse. Un nodo appartiene a un ciclo se e solo se appartiene a una componente fortemente connessa di dimensione almeno 2. I nodi aciclici sono quindi esattamente quelli che appartengono a componenti fortemente connesse di dimensione 1.

```
int acyclicNodes(GRAPH G)
```

```
  precondition:  $G.n \geq 0$ 
```

```
  int[] components = scc(G)
```

```
  int[] sizes = new int[0 .. G.n - 1] = {0}
```

```
  for  $u = 0$  to  $G.n - 1$  do
```

```
    | sizes[components[u]] = sizes[components[u]] + 1
```

```
  int count = 0
```

```
  for  $u = 0$  to  $G.n - 1$  do
```

```
    | if sizes[components[u]] == 1 then
```

```
      | count = count + 1
```

```
  return count
```

Il calcolo delle componenti fortemente connesse richiede tempo $\mathcal{O}(m+n)$. I cicli successivi scandiscono i nodi in tempo $\mathcal{O}(n)$. La complessità complessiva è quindi $\mathcal{O}(m+n)$. Questa soluzione ha preso 100%.

Errore tipico 1: confondere “raggiunge un ciclo” con “appartiene a un ciclo”. Consideriamo il grafo

$$a \rightarrow b, \quad b \rightarrow c, \quad c \rightarrow d, \quad d \rightarrow b.$$

Se si fa partire da a la funzione `hasCycleRec()` vista a lezione, la visita trova il ciclo formato da b, c, d e restituisce **true**. Questo però non implica che a sia ciclico: a raggiunge un ciclo, ma non esiste alcun cammino non vuoto che parte da a e ritorna in a . Per classificare il nodo di partenza bisogna controllare se la visita torna proprio al nodo di partenza, non se trova un arco all'indietro qualunque.

Errore tipico 2: riusare impropriamente un vettore *visited* globale. Nel problema della visita, *visited* serve a non visitare due volte lo stesso nodo. In questo problema, invece, ogni nodo di partenza pone una domanda diversa: “questo nodo appartiene a un ciclo?”. Quindi il fatto che un nodo sia già stato raggiunto in una visita precedente non significa che sia già stato contato o classificato correttamente.

Ad esempio, nel grafo aciclico

$$a \rightarrow b \rightarrow c,$$

una DFS partita da a marca come visitati tutti e tre i nodi. Se poi il ciclo esterno salta b e c solo perché risultano già visitati, l'algoritmo conta soltanto a , mentre i nodi aciclici sono tre. Un booleano *visited* globale non è sufficiente per memorizzare il risultato del problema; per riusare informazione tra visite servirebbe una vera classificazione dei nodi, come quella ottenuta tramite componenti fortemente connesse.

Esercizio B1 – Shakira – Punti ≥ 8

Il problema può essere risolto tramite backtracking, costruendo la scaletta da sinistra a destra. Il vettore $S[0 \dots m-1]$ contiene gli indici delle canzoni già scelte, mentre $used[song]$ indica se la canzone $song$ è già stata inserita nella scaletta.

setlists(*item*[] *type*, *int* *n*, *int* *m*)

precondition: $0 \leq m \leq n$

item[] *S* = **newitem**[$0 \dots m-1$]

boolean[] *used* = **new boolean**[$0 \dots n-1$] = **false**

slRec(*type*, *n*, *m*, *used*, *S*, 0)

slRec(*item*[] *type*, *int* *n*, *int* *m*, **boolean**[] *used*, *item*[] *S*, *int* *i*)

precondition: $0 \leq i \leq m$

if *i* == *m* **then**

print *S*

else

for *song* = 0 **to** *n* - 1 **do**

if *used*[*song*] == **false** **then**

boolean *valid* = **true**

if *i* ≥ 2 **and** *type*[*song*] == *type*[*S*[*i* - 1]] **and** *type*[*song*] == *type*[*S*[*i* - 2]] **then**

valid = **false**

if *valid* **then**

used[*song*] = **true**

S[*i*] = *song*

slRec(*type*, *n*, *m*, *used*, *S*, *i* + 1)

used[*song*] = **false**

L'invariante della chiamata ricorsiva è che $S[0 \dots i-1]$ contiene una scaletta parziale valida, composta da canzoni distinte. Ogni estensione usa solo canzoni non ancora inserite e viene scartata esattamente quando produrrebbe tre canzoni consecutive dello stesso tipo. Quindi ogni scaletta stampata è valida; viceversa, ogni scaletta valida non viene mai scartata e viene generata scegliendo le sue canzoni nello stesso ordine.

Senza potature, il numero di scalette di lunghezza m è $n!/(n-m)!$. Ogni nodo dell'albero di ricorsione esamina al più n canzoni e ogni stampa costa $\mathcal{O}(m)$; poiché $m \leq n$, un limite superiore semplice è

$$\mathcal{O}\left(n \cdot \frac{n!}{(n-m)!}\right).$$

Esercizio B2 – Corsi di laurea – Punti ≥ 10

Il problema può essere modellato come un problema di flusso massimo, in cui si assegna a ogni corso un professore. Per tenere traccia della qualità dell'assegnamento, distinguiamo fra assegnamenti *competenti* e *incompetenti*, e imponiamo per ciascun Corso di Laurea un numero minimo di corsi insegnati in modo competente.

Costruzione della Rete. Costruiamo una rete di flusso con i seguenti nodi:

- una sorgente s e un pozzo t ;
- un nodo per ogni professore p_j ;
- due nodi per ogni corso $c_i \in C$: ci_i e cc_i , che rappresentano il caso in cui c_i sia insegnato rispettivamente in modo *incompetente* o *competente*;

- per ogni Corso di Laurea $CdL_h \in CDLs$, due nodi $CdLi_h$ e $CdLc_h$, che raccolgono rispettivamente i corsi del CdL_h insegnati in modo incompetente e competente;
- un nodo CdL_h per ogni Corso di Laurea $CdL_h \in CDLs$.

Aggiungiamo i seguenti archi (tutti orientati da sinistra verso destra):

- (s, p_j) con capacità 3, per ogni professore p_j (ogni professore può insegnare al massimo 3 corsi);
- (p_j, cc_i) con capacità 1 se $c_i \in choices_j$, e (p_j, ci_i) con capacità 1 se $c_i \notin choices_j$: in questo modo, se il professore p_j insegna c_i , il flusso passa sul nodo corretto a seconda che l'insegnamento sia competente o no;
- $(ci_i, CdLi_h)$ con capacità 1 e $(cc_i, CdLc_h)$ con capacità 1, dove CdL_h è l'unico Corso di Laurea tale che $c_i \in courses_h$ (i piani didattici sono disgiunti, quindi tale h è ben definito);
- $(CdLi_h, CdL_h)$ con capacità $m_h - t_h$ e $(CdLc_h, CdL_h)$ con capacità m_h : il primo arco limita a $m_h - t_h$ il numero di corsi del CdL_h che possono essere insegnati in modo incompetente (forzando quindi che almeno t_h di essi siano insegnati in maniera competente), mentre il secondo non pone un vincolo stringente sui corsi insegnati in modo competente;
- (CdL_h, t) con capacità m_h , per ogni $CdL_h \in CDLs$.

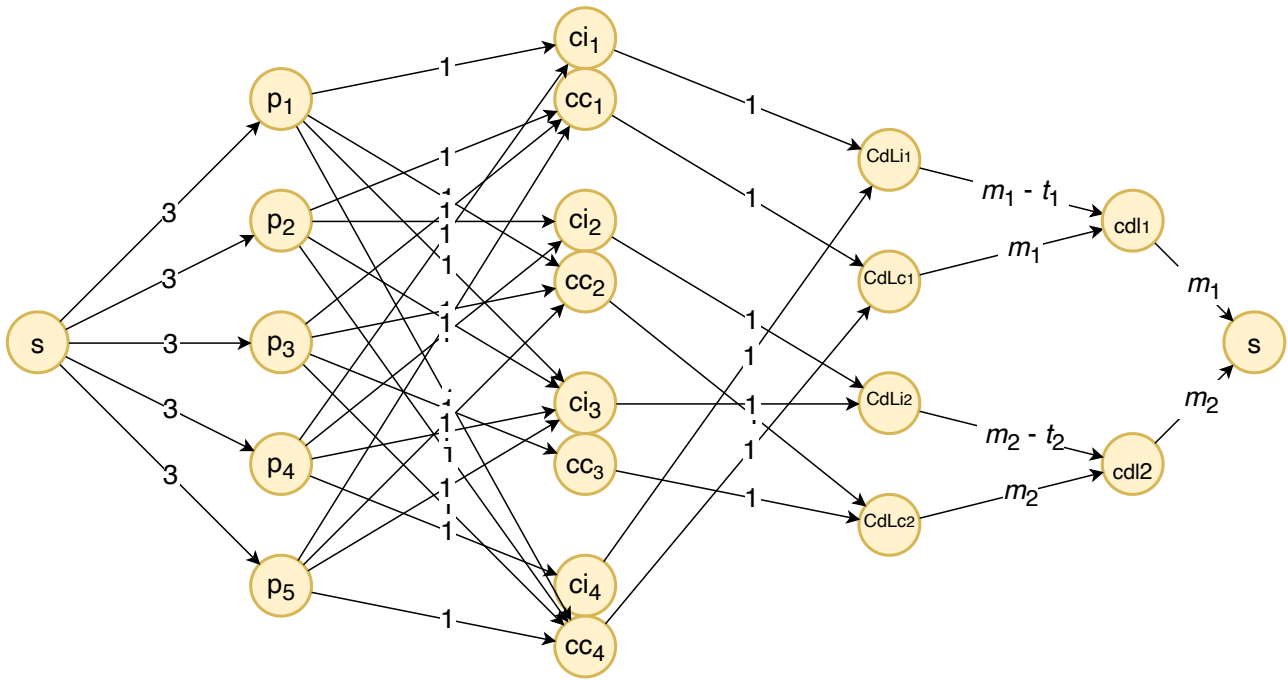
Correttezza (intuizione). Ogni unità di flusso rappresenta l'assegnamento di un professore a un corso.

- La capacità 3 sugli archi (s, p_j) impone che ogni professore insegni al massimo 3 corsi.
- La capacità 1 sugli archi in uscita dai professori garantisce che ogni unità di flusso corrisponda a un singolo corso.
- Poiché ogni corso c_i appartiene a un unico Corso di Laurea CdL_h , il flusso che passa da ci_i o cc_i confluisce necessariamente nei nodi del CdL_h .
- L'arco $(CdLi_h, CdL_h)$ con capacità $m_h - t_h$ forza che al più $m_h - t_h$ corsi del CdL_h siano insegnati in modo incompetente; equivalentemente, almeno t_h corsi devono essere insegnati in modo competente.
- Infine, l'arco (CdL_h, t) con capacità m_h impone che, per ciascun Corso di Laurea CdL_h , possano essere "contati" esattamente i suoi m_h corsi.

Ne segue che esiste un'assegnazione valida se e solo se il flusso massimo ha valore $|C| = \sum_{h=1}^k m_h$.

Complessità. Il numero di nodi è $|V| = 2 + n + 2m + 3k = \mathcal{O}(n + m + k)$. Il numero di archi è $|E| = n + \mathcal{O}(nm) + 2m + 3k = \mathcal{O}(nm + k)$. Il valore del flusso massimo è al più $|C| = m$ (ogni corso deve essere assegnato una volta). Usando Ford–Fulkerson con capacità intere, si ottiene complessità $\mathcal{O}(m(|V| + |E|)) = \mathcal{O}(m(nm + k))$.

Una possibile istanza con 5 professori, 4 corsi e 2 Corsi di Laurea è mostrata in Figura.



Esercizio B3 – Hateville PNRR – Punti ≥ 12

Il vincolo riguarda solo case adiacenti, quindi la scelta del colore per la casa i dipende unicamente dal colore scelto per la casa $i - 1$. Questo suggerisce programmazione dinamica.

Definiamo $DP[h][c]$ il costo minimo per ristrutturare le case $0 \dots h$, dipingendo la casa h col colore c . Allora:

$$DP[h][c] = \begin{cases} cost[0][c] & h = 0 \\ cost[h][c] + \min_{p \neq c} DP[h-1][p] & h > 0 \end{cases}$$

La risposta si trova nella riga $n - 1$ della matrice DP , scegliendo il valore minimo fra tutti i colori possibili.

```

int hateville(int[][] cost, int n, int k)
    precondition:  $n \geq 0$  and  $k \geq 2$ 
    if  $n == 0$  then
        return 0
    int[][] DP = newint[ $0 \dots n - 1$ ][ $0 \dots k - 1$ ]
    for  $c = 0$  to  $k - 1$  do
        DP[0][c] = cost[0][c]
    for  $h = 1$  to  $n - 1$  do
        for  $c = 0$  to  $k - 1$  do
            DP[h][c] =  $+\infty$ 
            for  $p = 0$  to  $k - 1$  do
                if  $p \neq c$  then
                    DP[h][c] = min(DP[h][c], DP[h-1][p] + cost[h][c])
    return min(DP[n-1])

```

La complessità è $\mathcal{O}(nk^2)$, per via dei cicli annidati.

Soluzione $\mathcal{O}(nk)$ In realtà, la scelta del colore migliore può essere realizzata memorizzando, per ogni casa h , il costo minimo e il secondo costo minimo fra tutti i colori possibili della riga precedente.

Se il colore corrente è diverso dal colore che realizza il minimo, si può usare il minimo; altrimenti bisogna usare il secondo minimo.

```

int hatevilleFast(int[][] cost, int n, int k)
    precondition:  $n \geq 0$  and  $k \geq 2$ 
    if  $n == 0$  then
        return 0
    int firstCost =  $+\infty$ 
    int secondCost =  $+\infty$ 
    int firstColor = -1
    for  $c = 0$  to  $k - 1$  do
        if  $cost[0][c] < firstCost$  then
            secondCost = firstCost
            firstCost =  $cost[0][c]$ 
            firstColor =  $c$ 
        else if  $cost[0][c] < secondCost$  then
            secondCost =  $cost[0][c]$ 
    for  $h = 1$  to  $n - 1$  do
        int newFirstCost =  $+\infty$ 
        int newSecondCost =  $+\infty$ 
        int newFirstColor = -1
        for  $c = 0$  to  $k - 1$  do
            int bestPrevious = iif( $c \neq firstColor, firstCost, secondCost$ )
            int value = bestPrevious +  $cost[h][c]$ 
            if  $value < newFirstCost$  then
                newSecondCost = newFirstCost
                newFirstCost = value
                newFirstColor =  $c$ 
            else if  $value < newSecondCost$  then
                newSecondCost = value
        firstCost = newFirstCost
        secondCost = newSecondCost
        firstColor = newFirstColor
    return firstCost

```

L'algoritmo mantiene esattamente le informazioni necessarie per calcolare la riga successiva della tabella *DP*, senza materializzare tutta la matrice. Ogni casa richiede una scansione dei k colori, quindi la complessità temporale è $\mathcal{O}(nk)$ e lo spazio aggiuntivo è $\mathcal{O}(1)$.